iscte
SINTRA
TECNOLOGIAS DIGITAIS
ECONOMIA E SOCIEDADE

Relatório do Projeto 1

Ilie Cosmin Iftime, nº 112779
Inês Rodrigues da Cruz, nº 123557
Sofia Feliciano Quintino, nº123554

Algoritmos Avançados de Procura
Docente: Ricardo Miranda

Licenciatura Tecnologias Digitais e Inteligência Artificial

2º Semestre 2025/2026

Índice

Introdução 1
Parte A- Puzzle Peças Deslizantes 1
Descrição dos algoritmos e heurísticas 1
Metodologia Experimental 2
Resultados e Análise 2
Discussão 3
Parte B- N-Rainhas 4
Descrição dos algoritmos 4
Metodologia Experimental 4
Resultados e Análise 5
Discussão 6
Parte C- Comparação A e B 7
Parte D- Simplex 8
Conclusão 10
Referências 11

Introdução

O presente relatório descreve a implementação e a análise empírica de diferentes metodologias de resolução de problemas. Numa primeira fase, exploram-se os algoritmos de procura clássica e heurística através do problema do 8-puzzle, avaliando o impacto das funções informadas na mitigação da explosão combinatória. Em seguida, a análise foca-se na procura local aplicada ao problema das N-Rainhas, testando a resiliência de diferentes métodos face a mínimos locais em configurações de elevada complexidade. Por último, recorre-se à Programação Linear e ao algoritmo Simplex para otimizar os recursos limitados de um restaurante, demonstrando a eficácia destas ferramentas matemáticas na maximização de resultados empresariais.

Parte A- Puzzle Peças Deslizantes

Descrição dos algoritmos e heurísticas

Para a resolução do problema do 8-puzzle, recorremos a diferentes estratégias de procura, divididas entre métodos não informados e informados, utilizando heurísticas para otimizar a eficiência da descoberta da solução.

Algoritmos de Procura Não Informada:

- Breadth-First Search (BFS): Explora o espaço de estados nível a nível. Garante a devolução da solução ótima em termos de número de movimentos, mas apresenta a desvantagem de ter um elevado custo de memória espacial.
- Iterative Deepening Search (IDS): Combina as vantagens da procura em profundidade (baixo consumo de memória) com as garantias da procura em largura (otimalidade da solução). O algoritmo funciona executando procuras em profundidade sucessivas, mas com um limite de profundidade que vai sendo incrementado iterativamente a cada tentativa até que o estado objetivo seja encontrado.

Procura Informada:

- Greedy Search: Seleciona o próximo estado baseando-se exclusivamente na heurística, expandindo o nó que parece estar mais próximo do objetivo. É habitualmente rápido e gasta pouca memória, mas não garante a solução ótima.
- A*: Combina o custo do caminho percorrido até ao momento, g(n), com o custo estimado até ao objetivo, h(n), minimizando a função objetivo $f(n) = g(n) + h(n)$. Sendo as heurísticas utilizadas admissíveis, este algoritmo garante a devolução da solução ótima, expandindo menos nós que os métodos não informados.

Heurísticas:

- Número de peças fora do lugar: Conta quantas peças não estão na sua posição final correta (excluindo a contagem do espaço vazio).
- Distância de Manhattan: Calcula a soma das distâncias absolutas verticais e horizontais de cada peça desde a sua posição atual até à sua posição objetivo.

Metodologia Experimental

A implementação dos algoritmos e a execução das respetivas experiências foram desenvolvidas na linguagem de programação Python. O estado do puzzle é representado computacionalmente como um tuplo de tuplos (uma matriz 3x3 imutável e *hashable*), o que permite a sua utilização direta em estruturas de dados como conjuntos e dicionários, garantindo um controlo eficiente dos estados já visitados. O valor zero indica o espaço vazio. Para assegurar a total reprodutibilidade dos resultados, os sucessores de cada estado são gerados assumindo uma ordem fixa para os movimentos do espaço vazio: cima, baixo, esquerda e direita.

O estado objetivo definido para todas as experiências foi a configuração sequencial padrão: 1 2 3 | 4 5 6 | 7 8 _. Para avaliar a robustez e eficiência dos algoritmos perante diferentes níveis de complexidade, definimos três estados iniciais com profundidades de solução distintas:

- Estado Fácil: 1 2 3 | 4 5 6 | _ 7 8
- Estado Médio: 1 2 3 | 5 _ 6 | 4 7 8
- Estado Difícil (Obrigatório): 8 6 7 | 2 5 4 | 3 _ 1. Este estado encontra-se particularmente distante do objetivo no espaço de estados, constituindo um cenário ideal para evidenciar as diferenças de desempenho e de complexidade computacional entre as várias abordagens.

Resultados e Análise

Nos estados com menor profundidade de solução (Fácil e Médio, com comprimentos ótimos de 2 e 4, respetivamente), todos os algoritmos apresentaram tempos de execução negligenciáveis (na ordem das dezenas de microssegundos). Contudo, é visível uma diferença no número de estados explorados entre a procura cega e a informada no estado médio: enquanto o BFS e o IDS exploraram 19 e 68 estados, os algoritmos com heurísticas (Greedy e A*) direcionaram a procura e precisaram apenas de explorar 5 estados. A discrepância entre o BFS e o IDS (19 vs 68 estados explorados) explica-se pelo funcionamento do *Iterative Deepening*, que revisita os mesmos nós das profundidades anteriores a cada nova iteração.

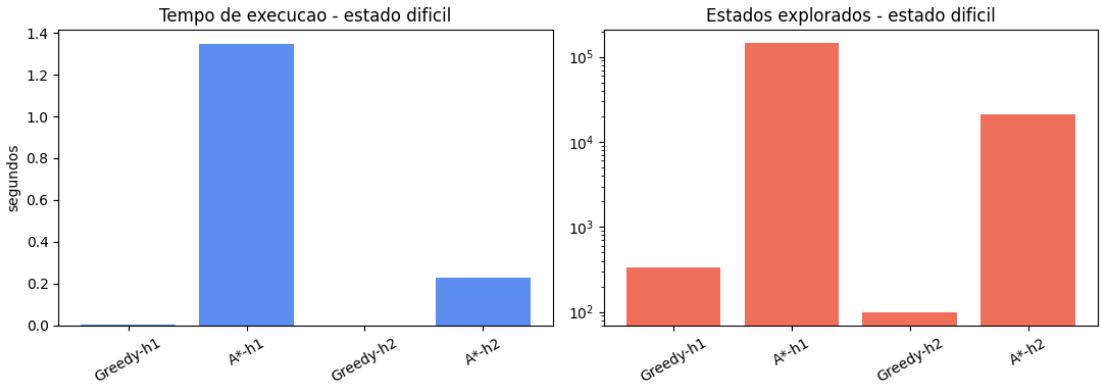
É no estado "Difícil" que as fragilidades e os pontos fortes de cada algoritmo ficam evidentes.

Não existem resultados para o BFS e IDS neste estado porque a complexidade de tempo e, sobretudo, a explosão do espaço de estados em memória esgotaram os recursos do sistema antes de encontrarem a solução de profundidade 31.

Analisando os dados, percebemos que o Greedy Search é extremamente rápido (menos de 0.003s) e explora muito poucos estados (100 a 337). No entanto, sacrifica a otimalidade da solução: devolveu caminhos de 63 e 47 movimentos em vez dos 31 movimentos ótimos. Isto acontece porque o Greedy é "cego" – olha apenas para a estimativa da heurística para o próximo passo, caindo frequentemente em caminhos subótimos. O A*, por outro lado, garantiu o caminho ótimo (31 movimentos) para ambas as heurísticas, ponderando o custo do caminho já percorrido.

Como evidenciado pelos gráficos gerados no projeto, existe uma diferença abismal de desempenho no A* consoante a heurística escolhida. O gráfico de barras dos tempos de execução mostra o A*-h1 (Peças fora do lugar) a demorar cerca de 1.35 segundos, enquanto o A*-h2 (Distância de Manhattan) demora menos de um quarto desse tempo

(~0.23s). Esta diferença é corroborada pelo gráfico logarítmico dos estados explorados: a heurística de Manhattan reduziu o número de estados expandidos de aproximadamente 143.000 para cerca de 21.000. Como a Distância de Manhattan é uma heurística mais "informada", permite ao A* descartar mais rapidamente ramos inúteis da árvore de procura.



Discussão

Qual o impacto da heurística no desempenho do A*? A heurística afeta drasticamente a eficiência do A*. No estado "difícil", a Distância de Manhattan (mais informada e dominante) reduziu os estados explorados de 143.849 para 21.198 e o tempo de 1.35s para 0.23s, face à heurística de peças fora do lugar.

A heurística “peças fora do lugar” é admissível? Justifique. Sim. Uma heurística é admissível se nunca sobrestimar o custo real até ao objetivo. Como cada peça mal posicionada requer pelo menos um movimento para chegar ao seu destino, o total de peças fora do lugar será sempre menor ou igual ao número real de movimentos necessários.

Em que situações o BFS deixa de ser viável? Porquê? O BFS torna-se inviável em soluções de elevada profundidade (como o estado "difícil", com 31 movimentos) devido à sua complexidade espacial exponencial (O(b^d)). Como armazena em memória todos os nós de um nível antes de avançar para o seguinte, esgota a memória RAM do sistema muito antes de encontrar a solução.

Compare Greedy vs A*: quando produzem soluções diferentes? O Greedy devolve frequentemente soluções subótimas (ex: 47 ou 63 movimentos no estado difícil) porque toma decisões baseando-se apenas na estimativa heurística h(n). Em contraste, o A* encontra a solução ótima (31 movimentos) porque minimiza f(n) = g(n) + h(n), contabilizando também o custo real já percorrido (g(n)), o que evita que se perca em caminhos subótimos longos.

O que acontece ao desempenho quando se passa de 8-puzzle para 15-puzzle? Embora o 15-puzzle não tenha sido implementado nos nossos testes práticos, a análise teórica e os estrangulamentos já observados no estado "difícil" do 8-puzzle permitem deduzir o impacto. Se o BFS e o IDS já esgotaram os recursos do nosso sistema para encontrar uma solução de 31 movimentos no 8-puzzle, seriam completamente impraticáveis no 15-puzzle. Da mesma forma, os tempos e a memória consumida pelo A* aumentariam drasticamente, tornando as heurísticas testadas insuficientes e exigindo heurísticas muito mais avançadas para que o problema fosse computacionalmente viável.

Existe algum algoritmo que garanta sempre solução ótima? Explique. Sim: BFS, IDS e A*. O BFS e o IDS exploram a árvore sistematicamente nível a nível, logo a primeira solução encontrada é obrigatoriamente a mais rasa (ótima). O A* garante a otimalidade de forma matemática desde que a heurística utilizada seja admissível, garantindo que nunca descarta caminhos potencialmente mais baratos.

Parte B- N-Rainhas

Descrição dos algoritmos

Por se tratar de um problema de otimização onde o caminho não é relevante, mas sim a configuração final, recorremos a algoritmos de procura local. Foram implementadas as seguintes abordagens:

- Hill Climbing**:** É um algoritmo de otimização local que avalia iterativamente as configurações vizinhas e move-se na direção de estados com maior qualidade (menor número de ataques entre rainhas). A sua principal limitação é a tendência para ficar preso em mínimos locais, falhando em encontrar a solução global caso não existam vizinhos estritamente melhores.
- Simulated Annealing: Este algoritmo permite escapar de mínimos locais ao aceitar transições para estados piores com uma determinada probabilidade. Essa probabilidade diminui progressivamente à medida que uma "temperatura" global baixa, tornando o algoritmo mais conservador nas fases finais da procura.
- Stochastic Beam Search: Em vez de manter apenas um estado na memória, este algoritmo rastreia múltiplos estados em simultâneo (definidos pelo parâmetro *beam width*). Ao selecionar estocasticamente os sucessores para a próxima geração, garante uma maior diversidade na exploração do espaço de estados, evitando a convergência prematura.
- Tabu Search: Uma extensão da procura local que utiliza uma estrutura de memória a curto prazo (a *tabu list*) para registar os últimos estados ou movimentos visitados. Ao proibir o retorno a esses estados durante um determinado número de iterações, força o algoritmo a explorar novas regiões do espaço de estados, facilitando a fuga de mínimos locais.

Metodologia Experimental

Para avaliar o comportamento e a escalabilidade dos algoritmos, o problema foi testado para diferentes dimensões de tabuleiro. Conforme exigido, foram definidos três cenários principais de complexidade crescente: N=8, N=20 e N=50.

Para garantir a fiabilidade estatística dos resultados, dado o carácter não determinístico da maioria dos métodos estudados, cada algoritmo estocástico foi executado 10 vezes independentes para cada configuração de tabuleiro.

Durante as execuções, procedeu-se à recolha das seguintes métricas de desempenho:

- Taxa de sucesso (percentagem de execuções que encontraram uma solução válida);
- Tempo médio de execução;
- Número médio de iterações até à paragem;
- Qualidade da solução final (medida pelo número de ataques residuais).

Adicionalmente, conduziu-se uma análise de sensibilidade para aferir o impacto dos hiperparâmetros no desempenho dos algoritmos, nomeadamente: a variação da temperatura no Simulated Annealing, o tamanho da lista tabu (tabu list size) no Tabu Search, e a largura do feixe (beam width) no Stochastic Beam Search.

Resultados e Análise

Para o tabuleiro clássico de 8 rainhas, todos os algoritmos demonstraram um bom desempenho. A taxa de sucesso variou entre 80% e 100%, com tempos de execução residuais (todos abaixo de meio segundo). O Tabu Search destacou-se imediatamente ao atingir 100% de sucesso no menor tempo médio (aprox. 0.0038s) e com o menor número de iterações (22.3).

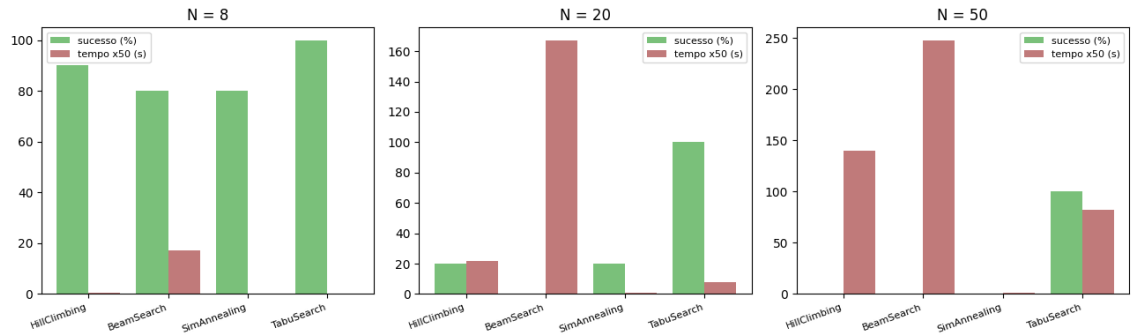
O aumento do tabuleiro revelou fragilidades drásticas na escalabilidade de quase todos os métodos, algo que é claramente visível nos gráficos de barras gerados.

O Hill Climbing viu a sua taxa de sucesso cair para 20% em N=20 e para 0% em N=50. Isto reflete a sua tendência natural para ficar preso em mínimos locais.

O Stochastic Beam Search foi o algoritmo com pior desempenho global. Em tabuleiros de N=20 e N=50, a taxa de sucesso foi nula (0%), os tempos de execução dispararam significativamente em comparação com os restantes (quase 5 segundos para N=50) e a qualidade da solução degradou-se severamente (média de 47 ataques residuais em N=50).

O Simulated Annealing revelou-se extremamente rápido por iteração, sendo capaz de executar milhares de iterações numa fração de segundo (aprox. 0.03s para N=50). No entanto, a taxa de sucesso também caiu para 0% em N=50, o que indica que, para instâncias maiores, os parâmetros de arrefecimento (temperatura inicial e taxa de decaimento) necessitariam de um ajuste para permitir uma exploração mais abrangente antes da convergência.

O Tabu Search foi, de forma incontestável, o algoritmo mais robusto desta experiência. Manteve uma taxa de sucesso imaculada de 100% em todas as dimensões testadas (N=8, 20 e 50), o que significa que encontrou sempre uma solução global sem qualquer ataque residual (Qualidade = 0.0). Apesar de o seu tempo de execução ter subido para aprox. 1.63s em N=50, fê-lo necessitando de apenas 40.4 iterações em média. O uso da memória a curto prazo (tabu list) impediu eficazmente que o algoritmo entrasse em ciclos infinitos de estados já visitados, forçando-o a escapar aos mínimos locais que capturaram o Hill Climbing.



Discussão

Porque é que o Hill Climbing falha frequentemente? O Hill Climbing falha frequentemente, em particular nas instâncias de maior dimensão (N=20 e N=50, onde a taxa de sucesso caiu para 20% e 0%, respetivamente), devido à sua enorme vulnerabilidade a mínimos locais. Sendo um algoritmo "guloso" que apenas aceita transições para estados vizinhos que melhorem ou mantenham a qualidade atual, acaba invariavelmente por ficar "preso" em configurações com ataques residuais onde nenhum movimento isolado melhora o estado do tabuleiro, falhando a solução ótima.

Como o Simulated Annealing evita mínimos locais? O Simulated Annealing escapa aos mínimos locais ao permitir, de forma estocástica, transições para estados piores (aumentando os ataques temporariamente). Esta probabilidade de aceitar movimentos subótimos é governada por uma "temperatura" global. Nas fases iniciais (temperatura alta), a probabilidade de aceitar piores estados é elevada, permitindo ao algoritmo explorar amplamente o espaço e saltar fora de vales subótimos. À medida que a temperatura desce, o algoritmo torna-se progressivamente mais rígido (semelhante ao Hill Climbing), convergindo para o mínimo global na zona onde se encontra.

Qual o impacto do beam width no Stochastic Beam Search? O parâmetro beam width (k) determina o número de estados mantidos simultaneamente na memória em cada geração. Um beam width maior promove uma exploração mais diversificada do espaço de procura, reduzindo a probabilidade de convergência prematura de todas as instâncias para o mesmo mínimo local (o que parece ter acontecido nos nossos testes, dada a taxa de sucesso de 0% para N>=20). Contudo, um k excessivamente grande aumenta drasticamente o custo de memória e de tempo de execução, como se comprova pelos quase 5 segundos exigidos no nosso N=50.

O Tabu Search melhora consistentemente os resultados? Justifique. Sim, de forma categórica. Nos resultados obtidos, o Tabu Search foi o único algoritmo a garantir um desempenho imaculado, com uma taxa de sucesso de 100% (zero ataques residuais) em todas as dimensões testadas. Esta consistência explica-se pela sua tabu list (memória a curto prazo), que proíbe o retorno a configurações ou movimentos recentes. Ao não poder retroceder nem entrar em ciclos infinitos, o algoritmo é forçado a desbravar novas áreas do espaço de procura, ultrapassando eficazmente os mínimos locais que "capturam" os outros algoritmos.

Qual algoritmo apresenta melhor compromisso entre tempo e qualidade? O Tabu Search apresenta, de forma inequívoca, o melhor compromisso. Embora o Simulated Annealing seja excecionalmente rápido a iterar (demorando cerca de 0.03s no total para N=50), apresenta uma qualidade péssima nessas dimensões (0% de sucesso). Por outro lado, o Tabu Search exigiu apenas cerca de 1.63 segundos e uma média de 40.4 iterações no tabuleiro N=50 para garantir uma qualidade absolutamente perfeita (100% de taxa de sucesso). É rápido o suficiente para uso prático e infalível na qualidade.

Como varia a dificuldade do problema com o aumento de N? Com o aumento de N, a dimensão do espaço de estados cresce exponencialmente. Isto comprova-se de forma flagrante nos nossos dados: num tabuleiro simples de N=8, algoritmos como o Beam Search, o Simulated Annealing e o Hill Climbing resolviam o problema com taxas de 80% a 90%. Quando passámos para N=20 e N=50, as taxas de sucesso destes mesmos algoritmos colapsaram para 20% e 0%, atestando a extrema dificuldade em navegar neste espaço ampliado sem mecanismos robustos de memória (como o Tabu Search).

Parte C- Comparação A e B

Ao longo deste projeto, analisámos dois paradigmas distintos de resolução de problemas, cada um com os seus benefícios e malefícios.

Na parte A (8-Puzzle), focado na procura em árvore/grafo:

- Procura Não Informada (BFS/IDS): Têm a vantagem de garantir a solução ótima (o caminho mais curto), mas a sua principal limitação é a complexidade espacial e temporal. Como exploram o espaço de forma "cega", tornam-se rapidamente impraticáveis em problemas com soluções profundas.
- Procura Informada (Greedy vs. A*): O Greedy Search tem a vantagem de ser rápido, mas a sua miopia (olhar apenas para a heurística) leva-o a soluções subótimas. Em contraste, o algoritmo A* surge como o método mais robusto deste grupo: ao combinar o custo real percorrido com a estimativa heurística, garante a otimalidade da solução (se a heurística for admissível) enquanto evita a explosão de memória do BFS.

Na parte B (N-Rainhas), focado na procura local:

- Hill Climbing: É extremamente simples e rápido, mas a sua principal limitação é a incapacidade de escapar a mínimos locais, o que resultou em taxas de sucesso muito baixas para tabuleiros maiores.
- Simulated Annealing e Beam Search: O primeiro introduz aleatoriedade para escapar de vales subótimos, enquanto o segundo explora múltiplos caminhos em simultâneo. Contudo, ambos mostraram forte dependência da afinação de hiperparâmetros (como a escalabilidade da temperatura ou a largura do feixe) para manterem eficácia em espaços de estados complexos.
- Tabu Search: Destacou-se como o melhor algoritmo desta categoria. A sua grande vantagem é o uso de memória a curto prazo (tabu list), que força a diversificação da procura e impede o regresso a estados recentes, contornando a principal limitação da procura local (os mínimos locais) com uma consistência notável (100% de sucesso).

A eficácia dos algoritmos de procura informada está intrinsecamente ligada à qualidade da função heurística. No 8-puzzle, a transição da heurística h1 (peças fora do lugar) para a heurística h2 (Distância de Manhattan) demonstrou de forma clara este impacto. Por ser mais informada e dominar a h1, a Distância de Manhattan permitiu ao algoritmo A* estimar o custo restante com maior precisão. Isto traduziu-se numa poda agressiva da árvore de procura, reduzindo o número de estados explorados em quase 85% e transformando uma execução morosa numa resolução quase instantânea. Uma boa heurística é, portanto, a chave para domar problemas de elevada complexidade.

A escolha entre métodos de procura clássica e procura local depende estritamente da natureza do problema:

- Quando usar cada abordagem? A procura clássica deve ser utilizada quando o caminho para a solução é o objetivo final (como no 8-puzzle, onde precisamos da sequência exata de movimentos) e quando é exigida uma solução matematicamente ótima. A procura local é a ferramenta ideal quando apenas o estado final interessa (como a disposição das rainhas no tabuleiro) e a otimalidade do percurso é irrelevante.

- Escalabilidade dos Métodos: A procura clássica sofre de problemas graves de escalabilidade. Devido ao crescimento exponencial do espaço de estados, passar de um 8-puzzle para um 15-puzzle inviabiliza métodos não informados e coloca enorme pressão de memória no A*. Por outro lado, os algoritmos de procura local escalam muito melhor em termos de uso de memória (mantendo apenas um ou poucos estados de cada vez). No entanto, à medida que a dimensão do problema cresce (ex: de N=8 para N=50 rainhas), a topologia do espaço de estados torna-se mais acidentada. A escalabilidade temporal e de sucesso destes métodos passa a depender inteiramente de mecanismos avançados de fuga a mínimos locais (como evidenciado pelo excelente desempenho do Tabu Search em tabuleiros grandes).

Parte D- Simplex

Nesta secção, abordamos o problema de gestão de produção de um restaurante que visa maximizar o seu lucro diário através da combinação ideal de três pratos, respeitando as limitações de recursos existentes.

1. Formulação do Problema

O objetivo é determinar as quantidades ideais de produção para:

- x1: Número de pratos de frango grelhado;
- x2: Número de pratos de massa vegetariana;
- x3: Número de pratos de salmão no forno.

A função objetivo a maximizar, que representa o lucro total, é definida por:

maxZ = 6x1 + 7x2 + 6x3

Sujeito às seguintes restrições de recursos diários:

- Tempo de preparação: x1 + 2x2 + x3 <= 60 unidades;
- Ingredientes frescos: 2x1 + x2 + 3x3 <= 95 unidades;
- Capacidade de empratamento: 2x1 + 2x2 + x3 <= 70 unidades;
- Não negatividade: x1, x2, x3 >= 0.

2. Passagem à Forma Standard

Para a aplicação do método Simplex, as desigualdades foram convertidas em igualdades através da introdução de variáveis de folga (s1, s2, s3), que representam a quantidade não utilizada de cada recurso:

- x1 + 2x2 + x3 + s1 = 60
- 2x1 + x2 + 3x3 + s2 = 95
- 2x1 + 2x2 + x3 + s3 = 70
- Z - 6x1 - 7x2 - 6x3 = 0

3. Resolução pelo Método Simplex

A resolução foi efetuada através de um algoritmo desenvolvido em Python, cujas etapas principais descrevemos de seguida.

O ponto de partida (solução básica inicial) assume a produção nula, onde x1=x2=x3=0 e as folgas absorvem toda a capacidade disponível (s1=60, s2=95, s3=70)

Base	x1	x2	x3	s1	s2	s3	b
Z	-6.0	-7.0	-6.0	0.0	0.0	0.0	0.0
s1	1.0	2.0	1.0	1.0	0.0	0.0	60.0
s2	2.0	2.0	3.0	0.0	1.0	0.0	95.0
s3	2.0	2.0	1.0	0.0	0.0	1.0	70.0

Detalhe da Primeira Iteração

- Variável de Entrada: Identificamos o coeficiente mais negativo na linha Z, que corresponde a x2 (-7.0). Esta variável entra na base pois é a que mais contribui para o aumento do lucro por unidade.
- Variável de Saída: Aplicamos o teste da razão mínima entre a coluna b e a coluna de x2:
  - Linha s1: 60 / 2 = 30
  - Linha s2: 95 / 1 = 95
  - Linha s3: 70 / 2 = 35

A menor razão (30) indica que a variável s1 sai da base.

- Pivotagem: O elemento pivot é 2.0. Após as operações elementares entre linhas para anular os restantes elementos da coluna de x2, obtemos um lucro parcial de 210€.

Iterações Seguintes (Resumo)

- Iteração 2: A variável x1 entra na base substituindo s3. O lucro sobe para 235€.
- Iteração 3: A variável x3 entra na base substituindo s2. O algoritmo atinge o ótimo pois deixam de existir coeficientes negativos na linha Z.~

4. Interpretação da Solução Ótima

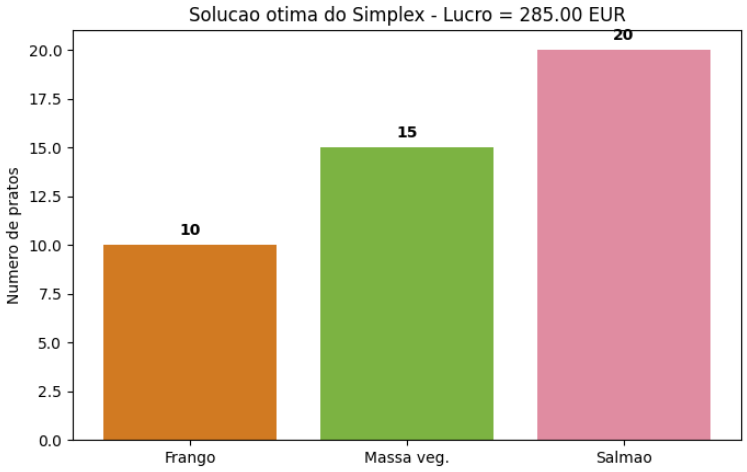
Após a execução do algoritmo, obtivemos os seguintes resultados finais.

Produção Ótima:

- Frango Grelhado (x1): 10 pratos
- Massa Vegetariana (x2): 15 pratos
- Salmão no Forno (x3): 20 pratos

Lucro Total Máximo: 285,00 €

A solução alcançada provou que é possível maximizar o lucro diário para 285 €, determinando com exatidão o mix de produção ideal (10 pratos de frango, 15 de massa e 20 de salmão). Mais do que calcular um valor financeiro, o modelo revelou um cenário de eficiência operacional perfeita, onde todo o tempo de preparação, ingredientes frescos e capacidade de empratamento são esgotados sem qualquer desperdício (variáveis de folga nulas no ótimo).



Conclusão

No Problema 1, a análise aos algoritmos de procura sublinhou que a escalabilidade é o principal desafio computacional. Na procura clássica (8-puzzle), demonstrou-se de forma prática que o algoritmo A*, quando munido de uma heurística informada e admissível (como a Distância de Manhattan), é a ferramenta ideal para domar a explosão combinatória do espaço de estados, garantindo a otimalidade sem esgotar a memória do sistema. Por outro lado, na procura local (N-Rainhas), confirmou-se que métodos "gulosos" como o Hill Climbing sucumbem facilmente a mínimos locais em problemas de maior dimensão. Para estes casos, mecanismos que forçam a diversificação e impedem a repetição de estados, como a tabu list do algoritmo Tabu Search, provaram ser indispensáveis, garantindo 100% de eficácia mesmo nos cenários mais complexos.

No Problema 2, a aplicação do método Simplex ilustrou o poder da Programação Linear na otimização da alocação de recursos físicos. Através de uma modelação matemática rigorosa das restrições do restaurante, foi possível determinar com exatidão a estratégia de produção diária que maximiza o lucro, provando ser uma ferramenta analítica fundamental para a tomada de decisão em contextos empresariais.

Em suma, os resultados obtidos ao longo das várias experiências reforçam uma lição central: não existe um "algoritmo universal". A escolha do método mais adequado, seja para descobrir o caminho ótimo num puzzle, posicionar peças num tabuleiro ou alocar ingredientes numa cozinha, deve ser sempre ditada pela natureza estrutural do problema, pelo objetivo pretendido e pelos recursos computacionais disponíveis.

Referências

[1] Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4.ª ed.). Pearson.

[2] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing.*Science*, 220(4598), 671–680.

[3] Glover, F. (1989). Tabu search — Part I. *ORSA Journal on Computing*, 1(3), 190–206.

[4] Dantzig, G. B. (1948). Programming of interdependent activities II. *Econometrica*, 17(3–4), 200–211.

[5] Korf, R. E. (1985). Depth-first iterative-deepening. *Artificial Intelligence*, 27(1), 97–109.